



POLITECNICO
MILANO 1863

Credit Card Customer Attrition

Arnaud Grassian

Politecnico di Milano

June 2026

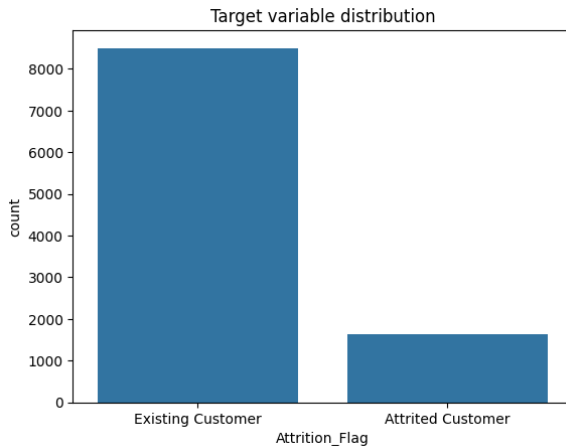
The problem

A bank wants to know which customers are about to close their credit card, so it can reach out before they leave.

- ▶ 10,127 customers
- ▶ 21 features per customer
- ▶ 16% have already left

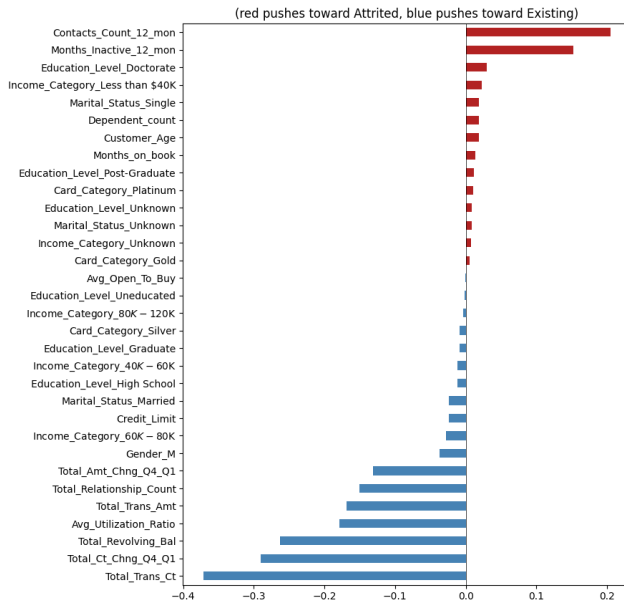
The model has to be predictive (catch churners early) and readable (the bank wants to see why a customer was flagged).

Target distribution



16% positive class : `class_weight='balanced'` for the linear and tree models,
`scale_pos_weight = 5.22` for XGBoost.

Correlation of each feature with the target



Methodology

EDA : histograms, boxplots by target and a correlation matrix all point to the same set of behavioural features (transaction count and amount, revolving balance, relationship count, Q4/Q1 change ratios).

Three approaches, all with hyperparameters tuned on validation :

- ▶ Lasso (L1) for feature selection, then Random Forest on the surviving features
- ▶ Random Forest on the full feature set
- ▶ XGBoost on the best set

Stratified 70 / 15 / 15 split, 5-fold stratified CV, scoring average_precision (AUPR).

Preprocessing

- ▶ One-hot encoding for all five categorical features
- ▶ StandardScaler fit on train only : no leakage from validation or test
- ▶ Lasso uses the standardized matrix (L1 penalizes feature scale). Random Forest and XGBoost use the raw matrix (tree models are scale-invariant)
- ▶ Class imbalance handled by reweighting, not by oversampling or undersampling :
class_weight='balanced' for Lasso and Random Forest, scale_pos_weight = 5.22 for XGBoost

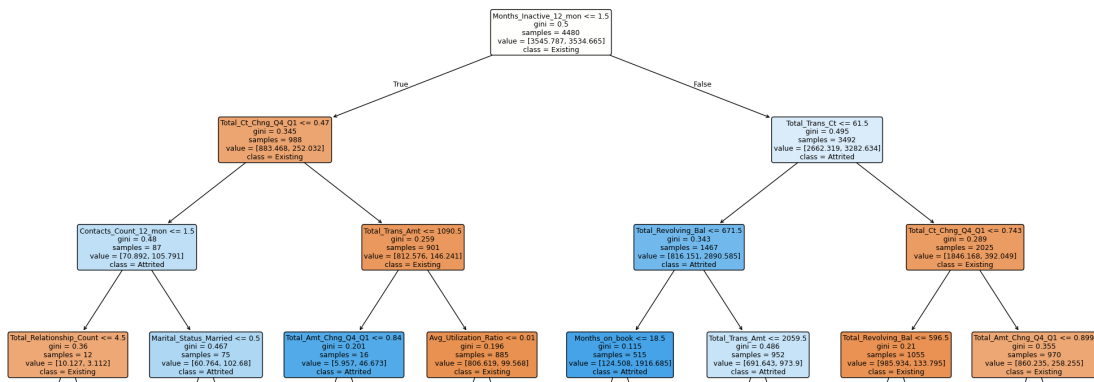
Step 1 - Lasso

Best regularization : $C = 0.0336$ (i.e. $\lambda = 29$). 21 features survive out of 32.

Feature	Coefficient
Total_Trans_Ct	-2.57
Total_Trans_Amt	+1.40
Total_Revolving_Bal	-0.57
Total_Relationship_Count	-0.55
Total_Ct_Chng_Q4_Q1	-0.54
Contacts_Count_12_mon	+0.49
Months_Inactive_12_mon	+0.46
Gender_M	-0.24

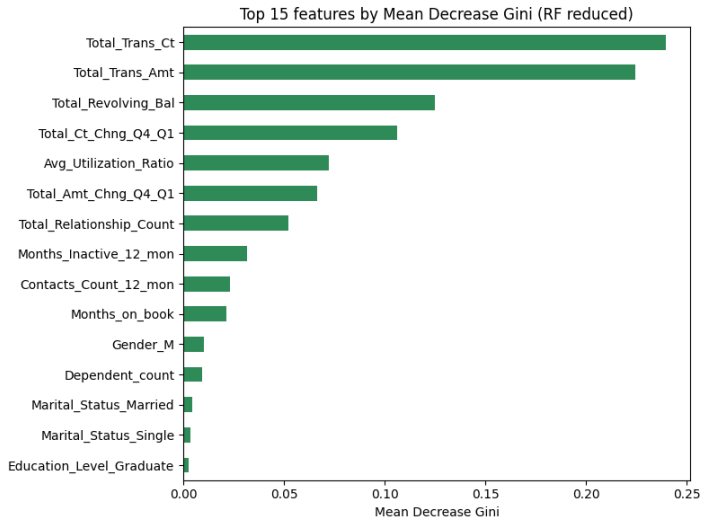
Top 8 features.

One tree of the Random Forest



Gini at the root is 0.5 because `class_weight='balanced'` weights the two classes equally. The root split is on `Months_Inactive_12_mon`, not on the top global feature, because each split picks from $\sqrt{21} = 5$ random features.

Random Forest - feature importance



Step 2 - Random Forest, reduced vs full

Same grid, same CV, same seed. Only the feature set differs.

	CV AUPR	Val AUPR
RF reduced (21 feat.)	0.930	0.920
RF full (32 feat.)	0.926	0.919

The Lasso step kept the model leaner without hurting performance.

Step 3 - XGBoost

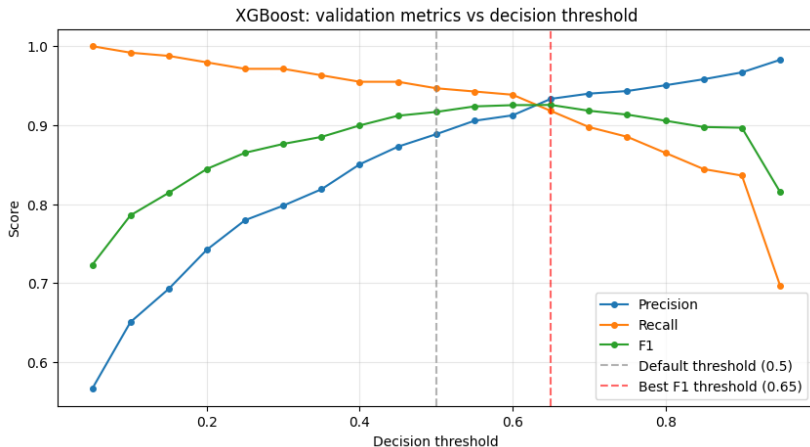
Trained on the Lasso-reduced set. Boosting-specific grid : `learning_rate`, `subsample`, `colsample_bytree`.

CV AUPR = 0.966 : beats the two Random Forests by four points.

Confusion matrix on validation, default threshold = 0.5 :

	Pred. Existing	Pred. Attrited
Actual Existing	1,246	29
Actual Attrited	13	231

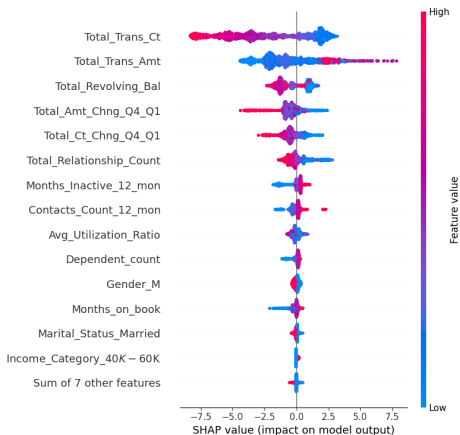
Threshold tuning on validation



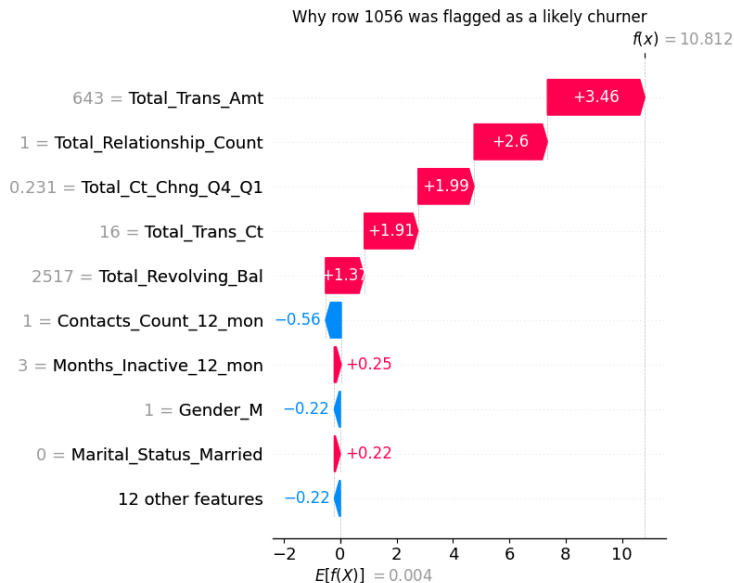
F1 peaks at threshold = 0.65. The default 0.5 is too conservative.

Step 4 - Interpretability

Four independent views of feature importance all agree on the same short list : Lasso coefficients, XGBoost built-in (average gain per split), permutation importance on validation, and SHAP values.



SHAP - one customer at a time



Step 5 - Business cost function

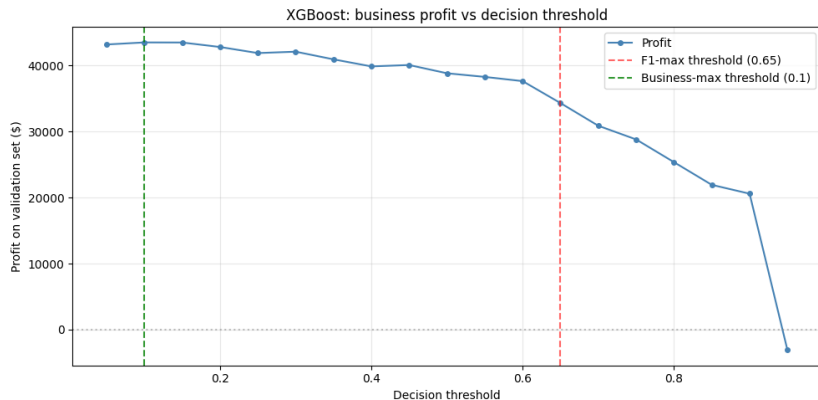
The default threshold 0.5 ignores the asymmetry between false alarms and missed churners.

$$\text{Profit} = \text{TP} \times 200 - \text{FP} \times 30 - \text{FN} \times 500$$

- ▶ 200 \$: net benefit of saving a churner
- ▶ 30 \$: cost of a false alarm
- ▶ 500 \$: cost of a missed churner

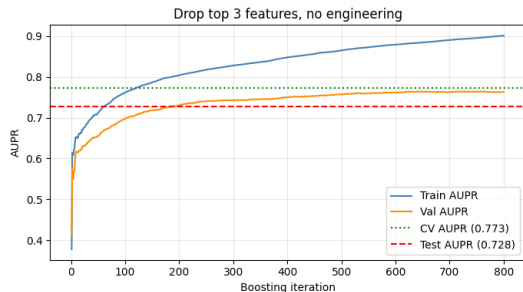
Sweep thresholds on validation, pick the one that maximises profit

Profit vs threshold - first XGBoost

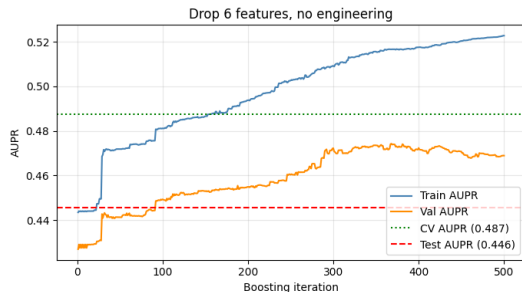


Optimum well below the F1-max value : missing a churner is too costly, so the model should flag aggressively.

What happens without engineering



Drop top 3 features. Test AUPR = 0.73



Drop 6 problematic features. Test AUPR = 0.45

Dropping these features without any replacement collapses AUPR.

Step 6 - A leakage-safe second pipeline

Some of the strongest features describe the customer's behaviour during the churn process, not before it.

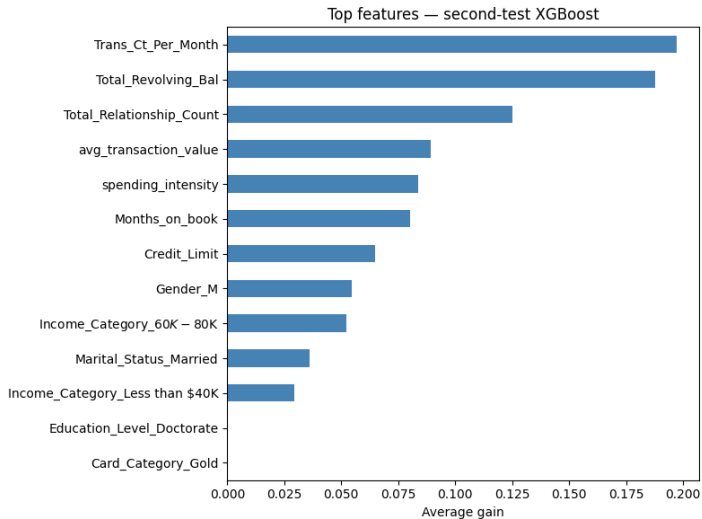
We drop the variables (6) that present time leaks and introduce three ratios in their place :

$$\text{Trans_Ct_Per_Month} = \frac{\text{Total_Trans_Ct}}{\text{Months_on_book}}$$

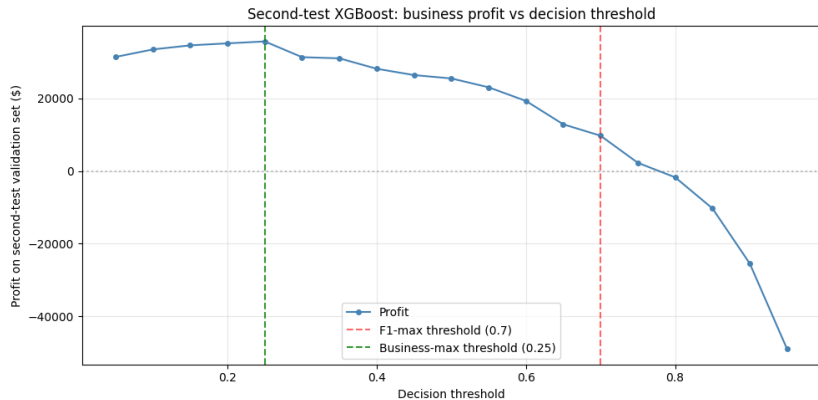
$$\text{avg_transaction_value} = \frac{\text{Total_Trans_Amt}}{\text{Total_Trans_Ct}}$$

$$\text{spending_intensity} = \frac{\text{Total_Trans_Amt}}{\text{Credit_Limit}}$$

Second-test XGBoost - feature importance

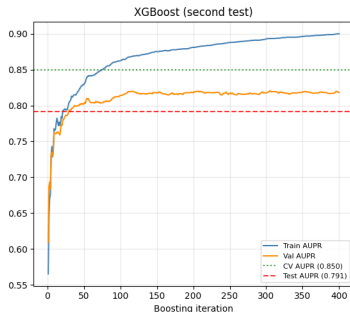
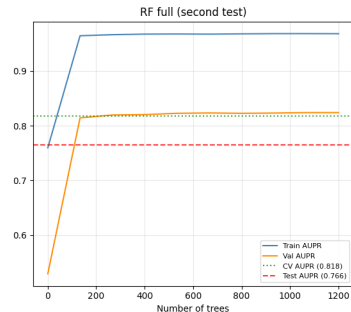
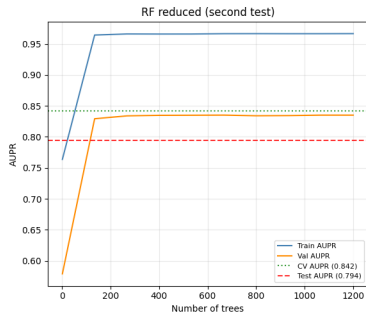
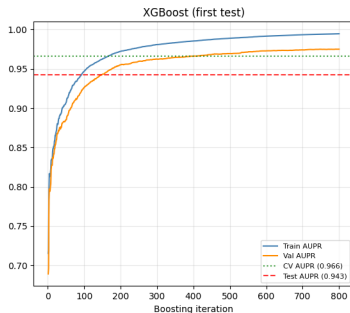
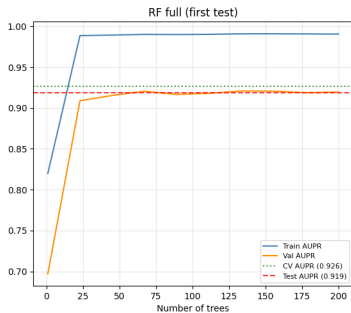
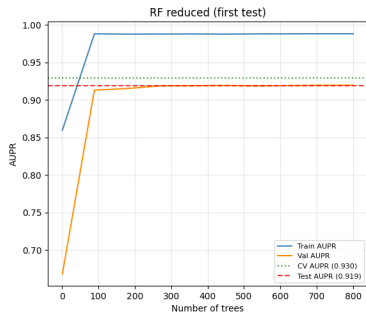


Profit vs threshold - second XGBoost



Lower precision, same recall : the operational cost of using only refreshable features.

Learning curves : first vs second test



Confusion matrices on the test set

At the F1-max threshold of each model. Test set : 1,520 customers (244 churners).

First XGBoost (t = 0.65)

	Pred. Existing	Pred. Attrited
Actual Existing	1,249	27
Actual Attrited	28	216

Recall 0.89 Precision 0.89

Second XGBoost (t = 0.70)

	Pred. Existing	Pred. Attrited
Actual Existing	1,197	79
Actual Attrited	58	186

Recall 0.76 Precision 0.70

The second model misses more churners (FN 58 vs 28) and produces about three times more false alarms : the price of leakage-safe features.

Test set - final numbers

	AUPR	Recall	Precision	Profit (\$)
XGBoost first test (business)	0.943	0.98	0.62	41,500
XGBoost second test (business)	0.791	0.95	0.48	32,750

A 21% profit drop is the measurable cost of removing the temporally leaky features.

Conclusion

XGBoost wins both rounds.

- ▶ Recommend deploying the second XGBoost for the monthly retention campaign : it uses features the bank can actually refresh
- ▶ Keep the first XGBoost as an internal benchmark
- ▶ Feature importance is stable across four different methods : the model is readable for a non-technical audience